Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

April 5, 2026

1 Home
2 Open-FDD engine
  2.1 Start here
  2.2 Contributors
  2.3 License
3 Getting started
4 Getting started
  4.1 Install
  4.2 Run rules on a DataFrame
  4.3 Clone and run tests (contributors)
  4.4 Full platform operators
  4.5 PyPI releases (maintainers)
5 Column map & resolvers
6 Column map & resolvers
  6.1 1. Plain dict (most IoT / warehouse flows)
  6.2 2. Brick TTL (`BrickTtlColumnMapResolver`)
  6.3 3. Manifest file (`ManifestColumnMapResolver`)
  6.4 4. Composite priority (`FirstWinsCompositeResolver`)
  6.5 Examples in the repo
  6.6 See also
7 Examples (repository)
8 Examples in the `open-fdd` repository
9 Engine-only & IoT pipelines
10 Engine-only and IoT-style use
  10.1 The only idea you need
  10.2 One `DataFrame` you can reuse
  10.3 Brick-style names (common default)
  10.4 Haystack-flavored names
  10.5 DBO / Google Digital Buildings–style names
  10.6 223P-scoped names
  10.7 Run **`RuleRunner`**

10.8 Manifests on disk, Brick TTL, composites
10.9 Docker `--mode engine` (different repo)
10.10 See also
11 Overview
12 Fault rules
12.1 Where rules live: config path and how to manage them
12.2 Hot reload (edit → run → view)
12.3 Rule types
12.4 YAML structure
12.5 Input resolution (100% Brick)
12.6 Running rules
12.7 Engineering metadata (223P-style) and energy-style analytics
12.8 Cookbook
13 Expression Rule Cookbook
14 Expression Rule Cookbook
14.1 100% Brick-model driven (no column in rules)
14.2 How to define expressions
14.3 AHU rules (reference-style)
14.3.1 Rule A — Duct static below setpoint at full fan speed
14.3.2 Rule B — Blended air temp below expected band
14.3.3 Rule C — Blended air temp above expected band
14.3.4 Rule D — Discharge air cold when heating commanded
14.3.5 Rule E — SAT too low with full heating
14.3.6 Rule F — SAT/MAT mismatch in economizer mode
14.3.7 Rule G — Ambient too warm for free cooling
14.3.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)
14.3.9 Rule I — Ambient vs blended mismatch (econ-only)
14.3.10 Rule J — Discharge above blended in cooling
14.3.11 Rule K — Discharge above setpoint in full cooling
14.3.12 Rule L — Cooling coil delta-T when inactive
14.3.13 Rule M — Heating coil delta-T when inactive
14.4 Central plant
14.4.1 Differential pressure at max pump speed
14.4.2 Plant flow high at max pump
14.4.3 Plant supply temp outside deadband
14.4.4 Chiller runtime over daily limit
14.5 Heat pumps
14.5.1 Discharge cold when heating
14.6 VAV zones
14.6.1 Excessive heating during warm weather
14.6.2 Damper or valve at full open
14.6.3 Zone and IAQ bounds

14.7 Opportunistic rules (economizer & ventilation)
14.7.1 Economizing when outdoor conditions are unfavorable
14.7.2 Mechanical cooling when econ could suffice
14.7.3 Low ventilation (estimated OA fraction)
14.7.4 Preheat over-conditioning
14.7.5 Blended air damper deviation
14.8 Weather station
14.8.1 Unrealistic temperature spike
14.8.2 RH bounds
14.8.3 Wind gust vs sustained
14.9 Sensor validation (bounds & flatline)
14.10 Follow-up (RFCs): multi-ontology selectors & cookbook matrix
15 Test bench rule catalog
16 Test bench rule catalog
16.1 Reference YAML files
16.2 Related docs
17 YAML rules → Pandas (under the hood)
18 YAML rules → Pandas DataFrames (under the hood)
18.1 1. YAML never becomes “one giant DataFrame of rules”
18.2 2. From Postgres rows to a site/equipment DataFrame
18.3 3. What `RuleRunner.run` does to that DataFrame
18.4 4. How a YAML rule dict becomes pandas expressions
18.5 5. Hot reload: YAML vs DataFrame lifetime
18.6 6. Mental model checklist
19 Fault rules for HVAC
20 Fault rules for HVAC
21 Contributing
22 Contributing to Open-FDD
22.1 Table of Contents
22.2 Code of Conduct
22.3 I Have a Question
22.4 I Want To Contribute
22.4.1 Legal notice
22.5 Reporting Bugs
22.5.1 Before submitting a bug report
22.5.2 How to submit a good bug report
22.6 Suggesting Enhancements
22.6.1 Before submitting
22.6.2 How to submit a good enhancement suggestion
22.7 Contributing Rules (Expression Rule Cookbook)
22.8 Your First Code Contribution
22.9 Improving the Documentation
22.10 Styleguides
22.11 Git workflow (branch → PR → sync)

22.12 Commit Messages
22.13 Join the Project Team
22.14 Attribution
23 Developer guide
24 Developer guide
24.1 Exported symbols
24.2 Repository layout
24.3 Setup and tests
24.4 Extending the engine
24.5 Other repository
25 Appendix
26 Appendix

Generated April 5, 2026

---

1 Home

2 Open-FDD engine

{: .fs-6 .fw-400 } **`open-fdd`** on PyPI is a small Python library: load **YAML** fault rules, run them on **pandas** `DataFrame`s, read boolean **`*_flag`** columns. If you already use notebooks, CSV exports, or a warehouse, you only need **`pip install open-fdd`**.

> **Full platform** — Docker Compose, API, BACnet, React UI, knowledge graph, `bootstrap.sh` — lives in **open-fdd-afdd-stack**.
> **Docs:** AFDD stack (GitHub Pages) · Repo

2.1 Start here

If you…	Read
Want the fastest path from install to `RuleRunner`	Getting started
Map Brick / ontology names to your column names	Column map & resolvers
Run rules on CSV or batch data (no database)	Getting started · Examples

If you…	Read
Author or tune YAML rules	Fault rules · Expression cookbook
Browse workshop-style examples	Examples in this repo
Use Docker `--mode engine` without the full UI	Engine-only / IoT

2.2 Contributors

Contributing · Developer guide · Tests (repo root)

2.3 License

MIT — open-fdd on GitHub.

3 Getting started

4 Getting started

You need **Python 3.9+** and **pip**. Basic **pandas** usage is enough to run rules.

4.1 Install

```
pip install open-fdd
```

Optional dependency groups (from `pyproject.toml`):

Extra	What it adds
`open-fdd[brick]`	**rdflib** — Brick `.ttl` → `column_map` via `BrickTtlColumnMapResolver`, TTL/SPARQL helpers
`open-fdd[viz]`	**matplotlib** — plots in notebooks or examples
`open-fdd[bacnet]`	

Extra	What it adds
	bacpypes3, **ifaddr**, **httpx** — BACnet-related scripts/examples
`open-fdd[dev]`	**pytest**, formatters, and stack-style deps used by the **open-fdd** test suite (contributors)

Other groups (**`[test]`**, **`[docx]`**, **`[platform]`**, **`[monitoring]`**) are for CI or specialized tooling; see **`pyproject.toml`** for the exact lists.

4.2 Run rules on a DataFrame

Rules are YAML (`type`, `inputs`, `params`, `flag`, ...). In YAML, **`inputs`** are keyed by **Brick-style names** (e.g. `Outside_Air_Temperature_Sensor`). Your `DataFrame` almost certainly uses **different column names** — pass **`column_map`** so each rule input points at the right column.

```
import pandas as pd
from open_fdd import RuleRunner

df = pd.DataFrame(
    {
        "timestamp": pd.date_range("2024-01-01", periods=24,
freq="h"),
        "oat": [72.0, 74.0, 95.0] + [55.0] * 21,  # one hot
stretch
    }
)

runner = RuleRunner(
    rules=[
        {
            "name": "oat_too_hot",
            "type": "expression",
            "flag": "oat_too_hot_flag",
            "inputs": {
                "Outside_Air_Temperature_Sensor": {
                    "brick": "Outside_Air_Temperature_Sensor",
                },
            },
            "params": {"hi": 90.0},
            "expression": "Outside_Air_Temperature_Sensor > hi",
        }
    ]
)

df_out = runner.run(
    df,
```

```
        timestamp_col="timestamp",
        column_map={"Outside_Air_Temperature_Sensor": "oat"},
    )
    # df_out includes oat_too_hot_flag (and original columns)
```

Load rules from disk instead of inline dicts:

```
    runner = RuleRunner("/path/to/dir/of/yaml/rules")
    df_out = runner.run(df, timestamp_col="timestamp",
column_map={...})
```

If a rule references columns you do not have, use **`skip_missing_columns=True`** so those rules are skipped instead of raising. More mapping options: Column map & resolvers. For CSV → `DataFrame`, use **`pandas.read_csv`** then **`runner.run(df, ...)`**; sample data and notebooks: Examples.

The **same YAML** runs in the full Docker platform; there the stack builds **`column_map`** from the data model. Here **you** supply the map (or **`open-fdd[brick]`** + a TTL file).

4.3 Clone and run tests (contributors)

```
    git clone https://github.com/bbartling/open-fdd.git
    cd open-fdd
    python3 -m venv .venv && source .venv/bin/activate
    pip install -U pip && pip install -e ".[dev]"
    pytest open_fdd/tests/ -v --tb=short
```

More detail: TESTING.md.

4.4 Full platform operators

Bootstrap, Compose services, Caddy, BACnet, REST APIs, and the React UI are documented in **open-fdd-afdd-stack**:

bbartling.github.io/open-fdd-afdd-stack

4.5 PyPI releases (maintainers)

Version bumps, tags, and the companion **`openfdd-engine`** package (same code, alternate install name) are covered in the stack docs:

PyPI releases (`open-fdd`)

5 Column map & resolvers

6 Column map & resolvers

YAML rules name their **`inputs`** with **Brick-style logical keys** (for example `Supply_Air_Temperature_Sensor`). Your **`pandas`** frame uses **your** column names (`sat`, `AHU1_SupplyTemp`, ...).

`column_map` is the bridge: `dict[str, str]` where each **key** is the **same string the rule YAML uses** (the logical / Brick-class side), and each **value** is the **actual `DataFrame` column name**.

Pass it to **`RuleRunner.run(..., column_map=...)`**. You can build that dict by hand, load it from a manifest, derive it from a Brick **`.ttl`** file, or **merge** several sources with a composite resolver.

6.1 1. Plain dict (most IoT / warehouse flows)

```
runner.run(
    df,
    column_map={
        "Supply_Air_Temperature_Sensor": "sat",
        "Outside_Air_Temperature_Sensor": "oat",
    },
)
```

No TTL file required if you already know the mapping. Keys must match the **input names** used in your rule YAML (see Fault rules overview).

6.2 2. Brick TTL (`BrickTtlColumnMapResolver`)

If you have a Brick **`.ttl`** model (same general idea as a stack deployment's RDF graph), **`BrickTtlColumnMapResolver`** runs the same SPARQL-based mapping logic the platform uses when resolving points from TTL.

```
from pathlib import Path
from open_fdd.engine.column_map_resolver import
BrickTtlColumnMapResolver
```

```
cm = BrickTtlColumnMapResolver().build_column_map(ttl_path=Path("model.ttl"))
runner.run(df, column_map=cm)
```

Install: **`pip install "open-fdd[brick]"`** (pulls **rdflib** and compatible **pyparsing** pins).

6.3 3. Manifest file (`ManifestColumnMapResolver`)

JSON or **YAML**: either a flat `logical_name → column_name` object, or a top-level **`column_map:`** nested mapping.

```
from pathlib import Path
from open_fdd.engine.column_map_resolver import ManifestColumnMapResolver

r = ManifestColumnMapResolver(Path("my_mapping.yaml"))
runner.run(df, column_map=r.build_column_map(ttl_path=Path(".")))
```

For a raw dict without the resolver class, use **`load_column_map_manifest(path)`** from the same module.

6.4 4. Composite priority (`FirstWinsCompositeResolver`)

Run multiple resolvers **in order**; for each logical key, the **first resolver that defines it wins** (for example: TTL for most points, manifest only for gaps).

```
from pathlib import Path
from open_fdd.engine.column_map_resolver import (
    BrickTtlColumnMapResolver,
    FirstWinsCompositeResolver,
    ManifestColumnMapResolver,
)

composite = FirstWinsCompositeResolver(
    BrickTtlColumnMapResolver(),
    ManifestColumnMapResolver("extras.yaml"),
)
```

```
        runner.run(df,
column_map=composite.build_column_map(ttl_path=Path("model.ttl")))
```

Security: There is **no** env-driven "import a resolver class by string" — compose resolvers in code or a small startup script.

6.5 Examples in the repo

Workshop — **`examples/column_map_resolver_workshop/`** (folder on GitHub):

Resource	Link
CLI (`brick` / `haystack` / `dbo` / `223p` / `minimal`)	`run_ontology_demo.py`
Walkthrough	`README.md`
Top-level index	`examples/README.md`
One-shot script (manifest + **`RuleRunner`**)	`demo_one_shot.py`
Multi-ontology illustration	`demo_multi_ontology_illustration.py`
Rules (Brick + Haystack + DBO + 223P)	`demo_rule.yaml`, `demo_rule_haystack.yaml`, `demo_rule_dbo.yaml`, `demo_rule_223p.yaml`
Manifests	`manifest_minimal.yaml`, `manifest_brick.yaml`, `manifest_haystack.yaml`, `manifest_dbo.yaml`, `manifest_223p.yaml` (illustrative slashes), `manifest_223p_safe.yaml` (expression-safe)

AHU / RTU notebooks and sample rules — **`examples/AHU/`** (folder on GitHub):

Resource	Link
Sample YAML rules	`rules/sensor_bounds.yaml`, `rules/sensor_flatline.yaml`, `rules/sat_operating_band.yaml`
Notebooks	

Resource	Link
	RTU11_standardized_refactored.ipynb, RTU7_standardized_refactored.ipynb, RTU7_machine_learning.ipynb
Sample CSV	RTU11.csv, AHU7.csv
Helpers	openfdd_notebook_helpers_v2.py

More narrative (Docker **`--mode engine`**, IoT pipelines, **`column_map` policy**): Engine-only & IoT pipelines. Curated doc links: Examples (repository).

6.6 See also

- Getting started — **`RuleRunner`** + **`column_map`** in one example
- Engine-only & IoT pipelines — integration patterns and resolver priority
- Fault rules overview — YAML **`inputs`** and Brick-oriented keys
- Expression rule cookbook — expression rules (same keying model)
- Examples (repository) — notebooks and CSVs under **`examples/AHU/`**

7 Examples (repository)

8 Examples in the `open-fdd` repository

All paths are under **github.com/bbartling/open-fdd/tree/master/examples**. Clone the repo, `pip install open-fdd` (or `pip install -e .`), then run from the **repo root**.

Fastest path — Brick, Haystack, DBO, or 223P column naming:

```
python examples/column_map_resolver_workshop/run_ontology_demo.py --list
python examples/column_map_resolver_workshop/run_ontology_demo.py haystack
```

Start here: **`examples/README.md`** on GitHub.

Area	What it shows
`column_map_resolver_workshop/`	**`run_ontology_demo.py`** (`brick`, `haystack`, `dbo`, `223p`, `minimal`) + manifests and paired rule YAML (workshop README)
`AHU/`	YAML rules under `rules/`, notebooks, sample CSVs (folder on GitHub)

Docker / **BACnet** / **API** examples and `bootstrap.sh` flows live with the platform: **open-fdd-afdd-stack**.

Install only what you need:

```
        pip install "open-fdd[brick,viz,dev]"   # notebooks + Brick helpers
```

9 Engine-only & IoT pipelines

10 Engine-only and IoT-style use

This page is for people who already have **time-series data** (historian export, MQTT pipeline, warehouse query, CSV) and want **`open-fdd`** to evaluate **YAML rules** on a **`pandas.DataFrame`**.

You do **not** need Postgres, TimescaleDB, or Docker for that path.

> **`from open_fdd import RuleRunner`** after **`pip install open-fdd`**. Rules live under **`open_fdd.engine`**; see Getting started for a full walkthrough.

10.1 The only idea you need

1. Build a **wide** table: one row per time step, one column per sensor (plus optional **`timestamp`**).
2. Your YAML rules talk in **logical names** (whatever you put in `inputs` and in **`expression`** — e.g. a Brick class name, a Haystack slug you invented, a 223P-scoped label).

3. **`column_map`** is a dict: **`{ logical_name: actual_column_name_in_df }`**. It tells **`RuleRunner`** which physical column to use when the rule says a given name.

The **left-hand keys** of **`column_map`** must match the names your **rule YAML** uses. The **right-hand values** are whatever your pipeline actually named the columns (`sat`, `AHU1_SA_T`, ...).

10.2 One `DataFrame` you can reuse

Most examples below use the same small frame: two sensors as columns **`sat`** and **`oat`**, plus **`timestamp`**.

```
import pandas as pd

df = pd.DataFrame(
    {
        "timestamp": pd.date_range("2025-01-01", periods=6,
freq="h", tz="UTC"),
        "sat": [72.0, 74.0, 76.0, 105.0, 70.0, 71.0],
        "oat": [35.0, 36.0, 38.0, 40.0, 34.0, 33.0],
    }
)
```

Your real job is whatever produces **`df`** (SQL, Parquet, MQTT aggregate, ...). **`open-fdd`** only cares that the columns exist.

10.3 Brick-style names (common default)

Rules often use **Brick class local names** like **`Supply_Air_Temperature_Sensor`**. Your warehouse columns stay short (**`sat`**, **`oat`**).

`column_map` (same as `manifest_brick.yaml`):

```
column_map = {
    "Supply_Air_Temperature_Sensor": "sat",
    "Outside_Air_Temperature_Sensor": "oat",
}
```

Use with rule YAML whose **`inputs`** and **`expression`** use those same strings. Example rule in the repo: `demo_rule.yaml`.

10.4 Haystack-flavored names

Haystack gives you **tags or refs**; you still choose **stable slugs** for rule logic. Here the logical keys are slug-style names; the frame columns stay **`sat`** / **`oat`**.

`column_map` (same shape as `manifest_haystack.yaml`):

```
column_map = {
    "discharge_air_temp_sensor": "sat",
    "outside_air_temp_sensor": "oat",
}
```

Your **rule YAML** must use **`discharge_air_temp_sensor`** (and **`outside_air_temp_sensor`** if needed) in **`inputs`** and in the **`expression`**, not the Brick names — unless you also change the map. One rule, one naming convention.

10.5 DBO / Google Digital Buildings–style names

Illustrative **type-style** keys (your site may differ). Physical columns are still **`sat`** / **`oat`**.

`column_map` (same as `manifest_dbo.yaml`):

```
column_map = {
    "SupplyAirTemperatureSensor": "sat",
    "OutsideAirTemperatureSensor": "oat",
}
```

Again: **rule YAML** uses these tokens in **`inputs`** / **`expression`**.

10.6 223P-scoped names

223P graphs often need **equipment** + **connection** in the label so “supply air temp” is not ambiguous. Your **graph** might use paths like **`AHU-1 / supply_air_temp`**; when you feed **`RuleRunner`**, you still emit a **`column_map`** from **whatever string you use as the rule input key** → **`sat`** / **`oat`**.

Illustrative map (same idea as `manifest_223p.yaml`):

```
column_map = {
    "AHU-1/supply_air_temp": "sat",
    "AHU-1/outside_air_temp": "oat",
}
```

Important — `type: expression`: Names in `inputs` become variables inside the `expression` string (via `pandas.eval` / `eval`). They must be **valid Python identifiers** (use `ahu1_supply_air_temp`, not `AHU-1/supply_air_temp`, in rules that use `expression`). `bounds` and `flatline` do not `eval` those keys the same way, so slash-heavy labels are less of a problem there — when in doubt, **normalize** graph labels to safe slugs for expression rules.

Example safe variant for the same physical columns:

```
column_map = {
    "ahu1_supply_air_temp": "sat",
    "ahu1_outside_air_temp": "oat",
}
```

10.7 Run `RuleRunner`

From a manifest file (JSON or YAML with a top-level `column_map:` or flat map):

```
from pathlib import Path
from open_fdd import RuleRunner
from open_fdd.engine.column_map_resolver import
load_column_map_manifest
from open_fdd.engine.runner import load_rule

root = Path("examples/column_map_resolver_workshop")  # adjust to
your clone
column_map = load_column_map_manifest(root /
"manifest_brick.yaml")
rules = [load_rule(root / "demo_rule.yaml")]
runner = RuleRunner(rules=rules)

out = runner.run(
    df,
    timestamp_col="timestamp",
    column_map=column_map,
    params={"units": "imperial"},
    skip_missing_columns=True,
)
```

Easiest — pick Brick, Haystack, DBO, or 223P from the shell: `run_ontology_demo.py` (e.g. **`python examples/column_map_resolver_workshop/run_ontology_demo.py haystack`**; use **`--list`** for modes).

Single-path script: `demo_one_shot.py` (minimal manifest + **`demo_high_sat_flag`**).

Static comparisons of ontology **key shapes** (no full rule run): `demo_multi_ontology_illustration.py`.

10.8 Manifests on disk, Brick TTL, composites

Covered step-by-step on **Column map & resolvers**:

- **YAML/JSON manifest** — `ManifestColumnMapResolver`, `load_column_map_manifest`
- **Brick `.ttl` file** — `BrickTtlColumnMapResolver` (**`pip install "open-fdd[brick]"`**)
- **TTL** + **small override file** — `FirstWinsCompositeResolver` (first resolver wins per key)

Implementation: `open_fdd/engine/column_map_resolver.py`. Workshop: **`examples/column_map_resolver_workshop/`** (README).

10.9 Docker `--mode engine` (different repo)

Postgres, `fdd-loop`, weather worker, and `./scripts/bootstrap.sh --mode engine` are part of the **open-fdd-afdd-stack** Docker platform, not something you run from the **`open-fdd`** library repo alone. That path is for teams who want **scheduled FDD inside Compose** with the same database pattern as the full product.

- **Modes and services:** Modular architecture (AFDD stack docs).
- **Bootstrap:** Getting started on the stack site.

If your data lives in **Snowflake, BigQuery, a lake, or flat files**, stay on **`RuleRunner`** + **`DataFrame`** above — no requirement to use that Docker slice.

10.10 See also

- Getting started — install, extras, one complete **`RuleRunner`** example
- Expression rule cookbook — authoring **`expression`** rules
- Examples (repository) — **`AHU/`** notebooks and CSVs
- Fault rules overview — YAML **`inputs`**, **`flag`**, types

Maintainers: alternate PyPI package name and release alignment — PyPI releases (`open-fdd`) (stack docs).

11 Overview

12 Fault rules

Fault rules are YAML-defined checks run against time-series DataFrames. Each rule produces a boolean fault flag. **Open-FDD is 100% Brick-model driven:** rule inputs refer only to Brick classes; column mapping comes from the Brick TTL via SPARQL and Brick external timeseries references (timeseries storage), which Open-FDD embraces at the heart of FDD.

> **Full Docker platform:** `rules_dir`, React Faults page, sync-to-DB, `POST /run-fdd`, and Grafana views are described for operators on the **AFDD stack docs — Fault rules**. This page stays focused on **YAML semantics** and the **library** path (`RuleRunner`).

12.1 Where rules live: config path and how to manage them

Single directory: All FDD rule YAML files live in one directory configured as **`rules_dir`** in platform config (RDF: `ofdd:rulesDir`, e.g. `"stack/rules"`). The FDD loop loads from this path every run; the rules API (list, upload, download, delete) uses the same path. **You still need `rules_dir` in config** — it defines where files are stored. The frontend does not replace it; it lets you manage the files *in* that directory.

Two ways to manage rules:

Method	Use case
React frontend (Faults page)	Upload new YAML, download existing files, delete files, and **Sync definitions** so the fault_definitions table updates without waiting for the next FDD run. Preferred when you have UI access.
Files on disk	Edit or add files directly under the configured path (e.g. `stack/rules/` on the host or in the container). Same outcome: next FDD run (or **Sync definitions** from the UI) picks them up.

Config: `rules_dir: "stack/rules"` (GET/PUT `/config` or `OFDD_RULES_DIR` at bootstrap). If that path does not exist, the loop falls back to `stack/rules`. Default rules ship in `stack/rules/` (e.g. `sensor_bounds.yaml`, `sensor_flatline.yaml`). See the Expression Rule Cookbook to add or adapt rules. For how YAML becomes pandas operations and how telemetry is pivoted into a DataFrame, see YAML rules → Pandas (under the hood).

Reference rule library (not loaded by default): Additional AHU, chiller, heat-pump, and weather YAML examples used for lab automation and docs live under **`openclaw/bench/rules_reference/`** in the repo. See the Test bench rule catalog for a table of every file with GitHub links.

12.2 Hot reload (edit → run → view)

Open-FDD is **AFDD** (Automated Fault Detection and Diagnostics). The project **supports hot reloading of YAML rule files** so the AFDD maintainer can tune faults without restarting. Rule YAML contains fault logic and **params** (thresholds, tolerances, windows); the FDD loop loads from `rules_dir` every run — no restart.

Fault definitions: Each FDD run syncs the loaded rules into the `fault_definitions` table (fault_id, name, category, equipment_types). When you add or edit a rule (via frontend upload or by editing a file in

`rules_dir`), the next run updates the DB and the Faults UI reflects the change. From the frontend you can also click **Sync definitions** to update the definitions table immediately.

1. **Add or edit** rules: use the Faults page (upload/paste YAML, or choose file) or edit files in `stack/rules/*.yaml`. Change `params` (e.g. `tolerance`, `rolling_window`) to tune sensitivity.
2. **Run** FDD: wait for the next scheduled run (per `rule_interval_hours` and `lookback_days` in platform config), or trigger with `touch config/.run_fdd_now` or `POST /run-fdd` (see Appendix: API Reference). Or use **Sync definitions** in the UI to only refresh the definitions table.
3. **View** fault results in the React Faults/Plots views or in Grafana (see Grafana SQL cookbook). Every run reloads all rules from disk — hot reload.

12.3 Rule types

Type	Purpose
`bounds`	Value outside `[low, high]`
`flatline`	Rolling spread < tolerance (stuck sensor)
`hunting`	Excessive state changes (PID hunting)
`expression`	Custom pandas/numpy expression
`oa_fraction`	OA fraction vs design airflow error
`erv_efficiency`	ERV effectiveness out of range

12.4 YAML structure

Inputs use **Brick class names** only; the Brick TTL (SPARQL) resolves each to a DataFrame column via external timeseries references. No `column` in YAML.

```yaml
name: sensor_bounds
type: bounds
flag: bad_sensor
inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
```

```yaml
        params:
          low: 40
          high: 90
          rolling_window: 6   # optional: require N consecutive True samples before flagging
```

Per-rule rolling window: Set `params.rolling_window` (e.g. `6`) in a rule to require that many consecutive True samples before the fault is flagged. Omit for "flag on any True." See `sensor_flatline.yaml`, `weather_temp_stuck.yaml`.

12.5 Input resolution (100% Brick)

- **Brick TTL:** Each rule input declares a Brick class (e.g. `brick:Supply_Air_Temperature_Sensor`). The engine resolves Brick points via SPARQL to their external timeseries reference (`ref:TimeseriesReference` / `ref:hasTimeseriesId`), yielding the DataFrame column name. See Brick timeseries storage.
- **Column map:** Built from the Brick model (e.g. `rdfs:label` or external_id) → column names; no `column` in rule YAML.

12.6 Running rules

- **Platform:** FDD loop loads rules from `rules_dir` (default `stack/rules`) each run; edit YAML and trigger a run to see changes in Grafana.
- **Standalone:** `RuleRunner(rules_path=...)` or `RuleRunner(rules=[...])`; call `run(df, ...)`.

12.7 Engineering metadata (223P-style) and energy-style analytics

FDD rules consume **time-series columns** built from the Brick graph (`ref:TimeseriesReference` + `ofdd:mapsToRuleInput`). **Engineering** data (design CFM, cooling tons, heating MBH, `s223:*` connection topology) lives on **equipment** in the DB and in the **same TTL** after import.

That split is intentional: the RuleRunner stays fast and pandas-centric. **Downstream**, you can combine:

- **Fault results** (which equipment, which fault, when), and
- **SPARQL** (or export TTL) for **rated capacity** and **topology** on that equipment, and
- **SQL** on `timeseries_readings` for duty estimates over the fault window

…to approximate **energy penalties** or rank impact; see Data model engineering (Brick + 223P MVP) on the stack docs for modeling patterns.

12.8 Cookbook

See Expression Rule Cookbook for AHU, chiller, weather, and advanced recipes.

13 Expression Rule Cookbook

14 Expression Rule Cookbook

A reference for building fault detection rules in open-fdd. Rules use **YAML** with **expression** type: when the expression evaluates to **True**, a fault is flagged. open-fdd injects **NumPy** as `np` into expression evaluation, so you can use `np.maximum`, `np.abs`, `np.sqrt`, etc. for vectorized math. **Tuning:** Change `params` in YAML and trigger an FDD run (or wait for the schedule); the platform hot-reloads rules each run, so no restart is needed. See Fault rules overview — Hot reload and Configuration on the AFDD stack docs (`rules_dir` is RDF-driven via GET `/config`).

> **`column_map` — platform vs library:** On the **full AFDD stack**, **`fdd-loop`** runs **`run_fdd_loop`**, which builds **`column_map`** from **`config/data_model.ttl`** using **`BrickTtlColumnMapResolver`** (same mapping as **`resolve_from_ttl`** in **`open_fdd.engine.brick_resolver`**). Recipes below stay **Brick-oriented** and assume that path. When you run **`RuleRunner`** yourself (**`pip install open-fdd`**, notebooks, external IoT pipelines), **you** supply **`column_map`** as a **dict** (logical / Brick-

> class keys → your DataFrame column names), or plug a custom **`ColumnMapResolver`** if you extend the platform loop. The rule YAML shape does not change; only **who** builds the map changes. See Getting started, Engine-only / IoT, and Column map & resolvers.

14.1 100% Brick-model driven (no column in rules)

Rule inputs use **Brick class names** only (e.g. `Supply_Air_Temperature_Sensor`). The **Brick TTL** is the source of truth: SPARQL resolves each Brick point to its external timeseries reference (`ref:TimeseriesReference` / `ref:hasTimeseriesId`), which yields the DataFrame column. Open-FDD embraces Brick timeseries storage at the heart of FDD — **rules never reference `column`**; the data model provides the mapping. See the SPARQL cookbook (Recipe 5) on the stack docs site; run similar queries from the **Data model** UI or API there.

- **Platform (DB):** Points have `brick_type` and `external_id`; TTL is built from DB. PATCH points or use data-model import to set `brick_type` (e.g. `Supply_Air_Temperature_Sensor` for SA-T).
- **Disambiguation:** When multiple points share a Brick class, use `ofdd:mapsToRuleInput` in TTL; the runner resolves `BrickClass|rule_input`.

14.2 How to define expressions

1. **Inputs** — Declare Brick classes only (`brick:Supply_Air_Temperature_Sensor`). The runner resolves these via the Brick TTL (SPARQL) and Brick external timeseries references to DataFrame columns. Do not add `column` in YAML.
2. **Params** — Thresholds and constants go in `params`. Reference by name (e.g. `err_thresh`, `vfd_max`).
3. **Expression** — Must evaluate to a boolean Series (True = fault). Use `&` (AND), `|` (OR), ~ (NOT). Use `.diff()`, `.rolling()`, `.notna()` for time-series logic.

Minimal example (TTL-driven; no column):

```yaml
name: high_temp_check
type: expression
flag: high_temp_flag

inputs:
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor

params:
  max_temp: 90.0

expression: |
  Supply_Air_Temperature_Sensor > max_temp
```

The Brick TTL (and SPARQL) maps each Brick class to the actual column via external timeseries references. Set `points.brick_type` in the DB and sync TTL so the engine can resolve columns from the Brick model.

14.3 AHU rules (reference-style)

The following rules follow common industry practice for air-handling fault detection. Rules A through M are adapted from ASHRAE Guideline 36 (GL36) AFDD guidance. Thresholds and logic are tunable; adjust params for your site.

14.3.1 Rule A — Duct static below setpoint at full fan speed

Static pressure under setpoint while supply fan runs near maximum. May indicate duct leakage, undersized fan, or terminal damper issues. *Adapted from GL36 AFDD guidance.*

```yaml
name: duct_static_low_at_full_speed
description: Static pressure below setpoint when fan at full speed
(GL36-inspired)
type: expression
flag: rule_a_flag
equipment_type: [VAV_AHU]

inputs:
  Supply_Air_Static_Pressure_Sensor:
    brick: Supply_Air_Static_Pressure_Sensor
  Supply_Air_Static_Pressure_Setpoint:
    brick: Supply_Air_Static_Pressure_Setpoint
  Supply_Fan_Speed_Command:
```

```
            brick: Supply_Fan_Speed_Command

        params:
          sp_margin: 0.12
          drv_hi_frac: 0.93
          drv_near_hi: 0.06

        expression: |
          (Supply_Air_Static_Pressure_Sensor <
Supply_Air_Static_Pressure_Setpoint - sp_margin) &
(Supply_Fan_Speed_Command >= drv_hi_frac - drv_near_hi)
```

14.3.2 Rule B — Blended air temp below expected band

Blended air temp should lie between outdoor and return. If below both (minus tolerance), suspect sensor or mixing fault. *Adapted from GL36 AFDD guidance.*

```
        name: blend_temp_below_band
        description: Blended air temp below expected range (OAT/RAT)
(GL36-inspired)
        type: expression
        flag: rule_b_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          rat_tol: 1.15
          oat_tol: 1.15

        expression: |
          (Mixed_Air_Temperature_Sensor - blend_tol <
np.minimum(Return_Air_Temperature_Sensor - rat_tol,
Outside_Air_Temperature_Sensor - oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)
```

14.3.3 Rule C — Blended air temp above expected band

Blended air temp above the higher of OAT and RAT (plus tolerance) indicates mixing or sensor fault. *Adapted from GL36 AFDD guidance.*

```
        name: blend_temp_above_band
        description: Blended air temp above expected range (OAT/RAT)
(GL36-inspired)
        type: expression
        flag: rule_c_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          rat_tol: 1.15
          oat_tol: 1.15

        expression: |
          (Mixed_Air_Temperature_Sensor - blend_tol >
np.maximum(Return_Air_Temperature_Sensor + rat_tol,
Outside_Air_Temperature_Sensor + oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)
```

Hunting/oscillation — see rule types (hunting).

14.3.4 Rule D — Discharge air cold when heating commanded

Discharge air temp below blended air when heating valve is open. Indicates heating coil or valve failure. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_cold_when_heating
        description: Discharge air below blended air when heating active
(GL36-inspired)
        type: expression
        flag: rule_d_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
```

```
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          sat_tol: 1.15
          fan_delta_t: 0.55

        expression: |
          (Supply_Air_Temperature_Sensor + sat_tol <=
Mixed_Air_Temperature_Sensor - blend_tol + fan_delta_t) &
(Valve_Command > 0.01) & (Supply_Fan_Speed_Command > 0.01)
```

OA fraction — see rule types (oa_fraction).

14.3.5 Rule E — SAT too low with full heating

Heating valve fully open but SAT remains below setpoint. Indicates undersized coil or valve failure. *Adapted from GL36 AFDD guidance.*

```
        name: sat_too_low_full_heating
        description: SAT below setpoint with heating valve fully open
(GL36-inspired)
        type: expression
        flag: rule_e_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Valve_Command:
            brick: Valve_Command
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          supply_err_thres: 1.0

        expression: |
          (Supply_Air_Temperature_Sensor < Supply_Air_Temperature_Setpoint
- supply_err_thres) & (Valve_Command > 0.9) &
(Supply_Fan_Speed_Command > 0)
```

14.3.6 Rule F — SAT/MAT mismatch in economizer mode

In economizer mode (min mechanical cooling), SAT should approximate MAT. Large deviation suggests coil bypass or sensor error. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_blend_mismatch_econ
        description: Discharge and blended air diverge in economizer mode
(GL36-inspired)
        type: expression
        flag: rule_f_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
            brick: Valve_Command

        params:
          fan_delta_t: 0.55
          blend_tol: 1.15
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (np.abs(Supply_Air_Temperature_Sensor - fan_delta_t -
Mixed_Air_Temperature_Sensor) > np.sqrt(sat_tol**2 +
blend_tol**2)) & (Damper_Position_Command > econ_min_open) &
(Valve_Command < 0.1)
```

14.3.7 Rule G — Ambient too warm for free cooling

Outside air temperature exceeds SAT setpoint while economizer is active and mechanical cooling is off. Economizer should not be providing “free” cooling under these conditions. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_warm_free_cool
        description: Outdoor air above setpoint in free cooling mode
(GL36-inspired)
        type: expression
        flag: rule_g_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
```

```
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
            brick: Valve_Command

        params:
          oat_tol: 1.15
          fan_delta_t: 0.55
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Outside_Air_Temperature_Sensor - oat_tol >
Supply_Air_Temperature_Setpoint - fan_delta_t + sat_tol) &
(Damper_Position_Command > econ_min_open) & (Valve_Command < 0.1)
```

14.3.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)

When both economizer and mechanical cooling are active, MAT should approach OAT. Large deviation suggests inadequate mixing or damper fault. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_blend_mismatch_econ_mech
        description: Outdoor and blended air diverge in econ+mech cooling
(GL36-inspired)
        type: expression
        flag: rule_h_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          oat_tol: 1.15
          blend_tol: 1.15

        expression: |
          (np.abs(Mixed_Air_Temperature_Sensor -
```

```
Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
oat_tol**2)) & (Valve_Command > 0.01) & (Damper_Position_Command >
0.9)
```

14.3.9 Rule I — Ambient vs blended mismatch (econ-only)

In economizer-only mode, MAT should match OAT. Deviation indicates damper or mixing fault. *Adapted from GL36 AFDD guidance.*

```yaml
        name: ambient_blend_mismatch_econ
        description: Outdoor and blended air diverge in economizer-only
mode (GL36-inspired)
        type: expression
        flag: rule_i_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          oat_tol: 1.15
          blend_tol: 1.15

        expression: |
          (np.abs(Mixed_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
oat_tol**2)) & (Damper_Position_Command > 0.9)
```

14.3.10 Rule J — Discharge above blended in cooling

SAT exceeds MAT when cooling (econ+mech or mech-only) is active. Indicates underperforming cooling coil or valve. *Adapted from GL36 AFDD guidance.*

```yaml
        name: discharge_above_blend_cooling
        description: Discharge air above blended air in cooling modes
(GL36-inspired)
        type: expression
        flag: rule_j_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
```

```
            brick: Mixed_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          fan_delta_t: 0.55
          blend_tol: 1.15
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Supply_Air_Temperature_Sensor > Mixed_Air_Temperature_Sensor +
np.sqrt(sat_tol**2 + blend_tol**2) + fan_delta_t) &
(((Damper_Position_Command > 0.9) & (Valve_Command > 0)) |
((Damper_Position_Command <= econ_min_open) & (Valve_Command >
0.9)))
```

14.3.11 Rule K — Discharge above setpoint in full cooling

SAT above setpoint with cooling at full capacity. Suggests undersized coil or plant limits. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_above_sp_full_cool
        description: Discharge air above setpoint in full cooling mode
(GL36-inspired)
        type: expression
        flag: rule_k_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Supply_Air_Temperature_Sensor > Supply_Air_Temperature_Setpoint
+ sat_tol) & (((Damper_Position_Command > 0.9) & (Valve_Command >
0.9)) | ((Damper_Position_Command <= econ_min_open) &
(Valve_Command > 0.9)))
```

14.3.12 Rule L — Cooling coil delta-T when inactive

Temperature drop across cooling coil when it should be off. Indicates leaking CHW valve or coil bypass. *Adapted from GL36 AFDD guidance.*

```
        name: clg_coil_drop_when_off
        description: Temperature drop across cooling coil when it should
be off (GL36-inspired)
        type: expression
        flag: rule_l_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Cooling_Coil_Entering_Air_Temperature_Sensor:
            brick: Cooling_Coil_Entering_Air_Temperature_Sensor
          Cooling_Coil_Leaving_Air_Temperature_Sensor:
            brick: Cooling_Coil_Leaving_Air_Temperature_Sensor
          Heating_Valve_Command:
            brick: Valve_Command
          Cooling_Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          enter_tol: 1.15
          leave_tol: 1.15
          econ_min_open: 0.12

        expression: |
          ((Cooling_Coil_Entering_Air_Temperature_Sensor -
Cooling_Coil_Leaving_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2)) & (((Heating_Valve_Command >
0) & (Cooling_Valve_Command == 0) & (Damper_Position_Command <=
econ_min_open)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command == 0) & (Damper_Position_Command >
econ_min_open)))
```

14.3.13 Rule M — Heating coil delta-T when inactive

Temperature rise across heating coil when it should be off. Indicates leaking HW valve. *Adapted from GL36 AFDD guidance.*

```
        name: htg_coil_rise_when_off
        description: Temperature rise across heating coil when it should
be off (GL36-inspired)
        type: expression
        flag: rule_m_flag
        equipment_type: [AHU, VAV_AHU]
```

```
        inputs:
          Heating_Coil_Entering_Air_Temperature_Sensor:
            brick: Heating_Coil_Entering_Air_Temperature_Sensor
          Heating_Coil_Leaving_Air_Temperature_Sensor:
            brick: Heating_Coil_Leaving_Air_Temperature_Sensor
          Heating_Valve_Command:
            brick: Valve_Command
          Cooling_Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          enter_tol: 1.15
          leave_tol: 1.15
          fan_delta_t: 0.55
          econ_min_open: 0.12

        expression: |
          ((Heating_Coil_Leaving_Air_Temperature_Sensor -
Heating_Coil_Entering_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2) + fan_delta_t) &
(((Heating_Valve_Command == 0) & (Cooling_Valve_Command == 0) &
(Damper_Position_Command > econ_min_open)) |
((Heating_Valve_Command == 0) & (Cooling_Valve_Command > 0) &
(Damper_Position_Command > 0.9)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command > 0) & (Damper_Position_Command <=
econ_min_open)))
```

Heat exchanger effectiveness — see rule types (erv_efficiency).

14.4 Central plant

14.4.1 Differential pressure at max pump speed

Variable-speed pump cannot meet differential pressure setpoint at full speed. Indicates piping issues, undersized pump, or blocked strainers.

```
        name: dp_below_sp_pump_max
        description: Differential pressure below setpoint with pump at
full speed
        type: expression
        flag: dp_pump_flag

        inputs:
          Differential_Pressure_Sensor:
            brick: Differential_Pressure_Sensor
          Differential_Pressure_Setpoint:
```

```
            brick: Differential_Pressure_Setpoint
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          dp_margin: 2.2
          pmp_hi_frac: 0.93
          pmp_near_hi: 0.06

        expression: |
          (Differential_Pressure_Sensor < Differential_Pressure_Setpoint -
dp_margin) & (Pump_Speed_Command >= pmp_hi_frac - pmp_near_hi)
```

14.4.2 Plant flow high at max pump

Flow unusually high with pump at high speed. Suggests short circuit or flow meter error.

```
        name: flow_high_pump_max
        description: Water flow unusually high with pump at full speed
        type: expression
        flag: flow_high_flag

        inputs:
          Water_Flow_Sensor:
            brick: Water_Flow_Sensor
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          flow_hi_limit: 1100.0
          pmp_hi_frac: 0.93
          pmp_near_hi: 0.06

        expression: |
          (Water_Flow_Sensor > flow_hi_limit) & (Pump_Speed_Command >=
pmp_hi_frac - pmp_near_hi)
```

14.4.3 Plant supply temp outside deadband

CHW supply temperature outside deadband while pump runs. Requires coil entering/leaving or plant supply temp sensors.

```
        name: plant_supply_temp_deadband
        description: Supply water temp outside deadband during pump
operation
        type: expression
        flag: chw_temp_fault

        inputs:
```

```
          Chilled_Water_Supply_Temperature_Sensor:
            brick: Chilled_Water_Supply_Temperature_Sensor
          Chilled_Water_Supply_Temperature_Setpoint:
            brick: Chilled_Water_Supply_Temperature_Setpoint
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          sp_band: 2.2

        expression: |
          (Pump_Speed_Command > 0.01) &
((Chilled_Water_Supply_Temperature_Sensor <
Chilled_Water_Supply_Temperature_Setpoint - sp_band) |
(Chilled_Water_Supply_Temperature_Sensor >
Chilled_Water_Supply_Temperature_Setpoint + sp_band))
```

14.4.4 Chiller runtime over daily limit

Chiller running beyond a daily threshold (e.g. 23 hours). Often indicates over-cooling or schedules that bypass lockout. Use a rolling window sized for your data interval (e.g. 5-min data → 276 samples ≈ 23 h).

```
        name: chiller_excessive_runtime
        description: Chiller runtime exceeds daily threshold (rolling
window)
        type: expression
        flag: chiller_runtime_fault

        inputs:
          Chiller_Status:
            brick: Chiller_Status

        params:
          # For 5-min data: 23 hours ≈ 276 samples; max_runtime = count of
"on" samples in window
          rolling_samples: 276
          max_runtime_samples: 264

        expression: |
          Chiller_Status.rolling(window=rolling_samples).sum() >
max_runtime_samples
```

Note: Adjust `rolling_samples` and `max_runtime_samples` for your data interval. For 5-min data, 264 samples ≈ 22 hours.

14.5 Heat pumps

14.5.1 Discharge cold when heating

Flags when discharge air temperature is below a minimum (e.g. 80°F) while the supply fan is running. Indicates the heat pump is not heating effectively—possible issues with compressor, refrigerant, or reversing valve. Tunable via `min_discharge_temp`. Logic: if zone temp < 69°F (heating mode), the discharge should be warm; a cold discharge with the fan on and a cold zone indicates the heat pump is failing to heat.

```
        name: hp_discharge_cold_when_heating
        description: Discharge air below minimum when fan on and zone is
cold (heating mode)
        type: expression
        flag: hp_discharge_cold_flag
        equipment_type: [Heat_Pump]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Zone_Temperature_Sensor:
            brick: Zone_Temperature_Sensor
          Supply_Fan_Status:
            brick: Supply_Fan_Status

        params:
          min_discharge_temp: 85
          zone_cold_threshold: 69.0
          fan_on_threshold: 0.01

        expression: |
          (Supply_Fan_Status > fan_on_threshold) &
(Zone_Temperature_Sensor < zone_cold_threshold) &
(Supply_Air_Temperature_Sensor < min_discharge_temp)
```

14.6 VAV zones

14.6.1 Excessive heating during warm weather

Reheat valve open when outdoor air is warm. Suggests over-cooling or setpoint issues.

```
        name: zone_reheat_warm_ambient
        description: Heating valve open when OAT is high
```

```yaml
        type: expression
        flag: excessive_heating_flag
        equipment_type: [VAV]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Reheat_Valve_Command:
            brick: Valve_Command

        params:
          t_amb_cutoff: 78.0
          reheat_open_min: 0.52

        expression: |
          (Outside_Air_Temperature_Sensor > t_amb_cutoff) &
(Reheat_Valve_Command > reheat_open_min)
```

14.6.2 Damper or valve at full open

Damper or reheat valve consistently at full open. Indicates override, undersized equipment, or control fault.

```yaml
        name: zone_damper_valve_full_open
        description: Damper or valve at full open for extended period
        type: expression
        flag: damper_100_flag
        equipment_type: [VAV]

        inputs:
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          full_open_pct: 97.5
          roll_samples: 105

        expression: |
          (Damper_Position_Command > full_open_pct) &
(Damper_Position_Command.rolling(roll_samples).min() >
full_open_pct)
```

14.6.3 Zone and IAQ bounds

For CO2 and zone temperature out-of-range checks, use the bounds rule type — `co2_bounds` and `zone_temp_bounds` examples.

14.7 Opportunistic rules (economizer & ventilation)

14.7.1 Economizing when outdoor conditions are unfavorable

Economizer active when outdoor air is too warm or humid. Use OAT (or enthalpy if available) vs. threshold.

```
        name: econ_active_warm_ambient
        description: OA damper open when outdoor conditions do not favor
economizing
        type: expression
        flag: econ_when_shouldnt_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          t_amb_econ_cutoff: 63.0
          dpr_econ_min: 0.42

        expression: |
          (Outside_Air_Temperature_Sensor > t_amb_econ_cutoff) &
(Damper_Position_Command > dpr_econ_min)
```

14.7.2 Mechanical cooling when econ could suffice

Cooling valve open when outdoor air is cool enough for economizer. Opportunity to reduce mechanical cooling.

```
        name: mech_cool_when_econ_available
        description: Mechanical cooling active when economizing could
suffice
        type: expression
        flag: cooling_when_econ_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
```

```
            brick: Valve_Command

        params:
          t_amb_econ_cutoff: 63.0
          dpr_not_econ_max: 0.32

        expression: |
          (Outside_Air_Temperature_Sensor < t_amb_econ_cutoff) &
(Damper_Position_Command < dpr_not_econ_max) & (Valve_Command >
0.01)
```

14.7.3 Low ventilation (estimated OA fraction)

For units without airflow meters, estimate OA fraction from OAT, MAT, RAT. Flag when below minimum design OA.

```
        name: low_oa_fraction_estimated
        description: Estimated OA fraction below minimum (OAT, MAT, RAT
method)
        type: expression
        flag: low_vent_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          oa_min_pct: 21.0
          t_rat_oat_min_gap: 2.2

        expression: |
          (Supply_Fan_Speed_Command > 0.01) &
(np.abs(Return_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > t_rat_oat_min_gap) &
(((Mixed_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) /
(Outside_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) *
100) < oa_min_pct)
```

Note: Guard against division-by-zero when outdoor and return temps are close.

14.7.4 Preheat over-conditioning

Preheat coil leaving temp higher than needed (e.g. above OAT when OAT > SAT SP, or above SAT SP when OAT < SAT SP). Indicates wasted heating energy.

```yaml
name: preheat_excess_temp
description: Preheat coil leaving temp above required level
type: expression
flag: preheat_waste_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Preheat_Coil_Leaving_Air_Temperature_Sensor:
    brick: Preheat_Coil_Leaving_Air_Temperature_Sensor
  Supply_Air_Temperature_Setpoint:
    brick: Supply_Air_Temperature_Setpoint
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Valve_Command:
    brick: Valve_Command

params:
  excess_tol: 2.2

expression: |
  (Valve_Command > 0.01) & (((Outside_Air_Temperature_Sensor >
Supply_Air_Temperature_Setpoint) &
(Preheat_Coil_Leaving_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor > excess_tol)) |
((Outside_Air_Temperature_Sensor <
Supply_Air_Temperature_Setpoint) &
(Preheat_Coil_Leaving_Air_Temperature_Sensor -
Supply_Air_Temperature_Setpoint > excess_tol)))
```

14.7.5 Blended air damper deviation

Expected MAT (from OAT, RAT, damper positions) differs from measured MAT. Indicates damper leakage or faulty mixing.

Requires airflow or damper position data; logic is more involved. A simplified version compares MAT to a weighted blend of OAT and RAT when dampers suggest significant OA.

14.8 Weather station

weather_temp_stuck (flatline) — see flatline rule type.

14.8.1 Unrealistic temperature spike

Temperature change between consecutive readings exceeds physical limit.

```yaml
name: weather_temp_spike
description: Unrealistic temperature change between readings
type: expression
flag: fault_temp_spike

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor

params:
  spike_limit: 16.0

expression: |
  Outside_Air_Temperature_Sensor.diff().abs() > spike_limit
```

14.8.2 RH bounds

For relative humidity out-of-range, use the bounds rule type — `rh_bounds` example.

14.8.3 Wind gust vs sustained

```yaml
name: weather_gust_lt_wind
description: Wind gust reported lower than sustained wind
type: expression
flag: fault_gust_lt_wind

inputs:
  Wind_Gust_Speed_Sensor:
    brick: Wind_Gust_Speed_Sensor
  Wind_Speed_Sensor:
    brick: Wind_Speed_Sensor

expression: |
  Wind_Gust_Speed_Sensor.notna() & Wind_Speed_Sensor.notna() &
(Wind_Gust_Speed_Sensor < Wind_Speed_Sensor)
```

14.9 Sensor validation (bounds & flatline)

Use the bounds and flatline rule types for generic sensor checks. Typical bounds:

Sensor type	Min	Max
Zone temp	40	100
Supply air temp	40	150
Air pressure (inH2O)	-5	10
RH	0	100
Chilled water temp	35	100
Hot water temp	50	212
Condenser water	50	110
CO2 (ppm)	400	2000

14.10 Follow-up (RFCs): multi-ontology selectors & cookbook matrix

Multi-ontology rule YAML (e.g. optional `selectors.brick` / `.haystack` / `.dbo` / `.s223` under each input) is a **separate RFC** — it would feed the same **`column_map`** / resolver layer while the **full AFDD stack stays Brick-only by default** for v1. Track design discussion on GitHub **#122** and follow-on issues.

Cookbook matrix / generator: When a selector schema is stable, we can add a **generated appendix** (logical input × Brick × Haystack × DBO × 223P notes) from a single CSV/YAML source so the long cookbook does not drift. Until then, see **`examples/column_map_resolver_workshop/`** for manifest-based illustrations.

Next: Fake fault schedule / flatline monitoring (AFDD stack operators).

15 Test bench rule catalog

16 Test bench rule catalog

The **AFDD stack** ships default rules under `stack/rules/` (e.g. `sensor_bounds.yaml`, `sensor_flatline.yaml`). For **extended HVAC, chiller, heat-pump, and weather expression rules** used in lab automation and cookbook examples, the stack repo keeps a **reference set** under **`openclaw/bench/rules_reference/`** (not loaded until you copy or upload them).

Each file below links to the copy on **GitHub** (`master` branch) so the published docs site stays in sync with the repo.

16.1 Reference YAML files

File	GitHub
`ahu_fc1.yaml`	link
`ahu_fc2.yaml`	link
`ahu_fc3.yaml`	link
`ahu_fc4.yaml`	link
`ahu_fc5.yaml`	link
`ahu_fc6.yaml`	link
`ahu_fc7.yaml`	link
`ahu_fc8.yaml`	link
`ahu_fc9.yaml`	link
`ahu_fc10.yaml`	link
`ahu_fc11.yaml`	link
`ahu_fc12.yaml`	link
`ahu_fc13.yaml`	link
`ahu_fc14.yaml`	link
`ahu_fc15.yaml`	link
`ahu_fc16.yaml`	link
`ahu_rule_a.yaml`	link
`chiller_flow_fc2.yaml`	link

File	GitHub
`chiller_pump_fc1.yaml`	link
`hp_discharge_cold_when_heating.yaml`	link
`sensor_bounds.yaml`	link
`sensor_flatline.yaml`	link
`weather_gust_lt_wind.yaml`	link
`weather_rh_out_of_range.yaml`	link
`weather_temp_spike.yaml`	link
`weather_temp_stuck.yaml`	link

16.2 Related docs

- Fault rules overview — where live rules live (`stack/rules`) and how to upload or sync.
- Expression rule cookbook — AHU, chiller, weather patterns that match many of these files.
- OpenClaw lab README — bench layout including `rules_reference/` and E2E automation.

The former **open-fdd-automated-testing** repo is superseded by the **`openclaw/`** tree in **open-fdd-afdd-stack**; do not link automation to a separate rules tree.

17 YAML rules → Pandas (under the hood)

18 YAML rules → Pandas DataFrames (under the hood)

This note is for engineers who want to see **exactly** how Open-FDD turns **rule YAML on disk** into **pandas operations** on **time-series DataFrames**—and where Brick TTL fits in. It complements Fault rules overview, Getting started, and Examples.

18.1 1. YAML never becomes “one giant DataFrame of rules”

Rule files are **configuration**, not tabular data. Open-FDD:

1. Reads each `*.yaml` in `rules_dir` with PyYAML (`yaml.safe_load`) into a **Python `dict`** per file (`open_fdd.engine.runner.load_rule` / `load_rules_from_dir`).
2. Keeps those dicts in a **`list[dict]`** inside `RuleRunner` (`RuleRunner(rules=...)` or `RuleRunner(rules_path=...)`).
3. At evaluation time, walks that list and, for each rule, computes a **boolean mask** (`pandas.Series`) aligned to the **sensor DataFrame’s index**, then writes a **new column** for the fault flag (e.g. `my_rule_flag`).

So: **rules** = **list of dicts in memory**; **data** = **one wide-ish DataFrame** of timestamps and point columns.

18.2 2. From Postgres rows to a site/equipment DataFrame

The continuous loop (`openfdd_stack.platform.loop.run_fdd_loop`) loads telemetry with SQL, then reshapes it for pandas:

1. **Query** `timeseries_readings` joined to `points` for the time window (`load_timeseries_for_site` / `load_timeseries_for_equipment`).
2. **Build a long table**: `pd.DataFrame(rows)` with columns like `ts`, `external_id`, `value`.
3. **Pivot to wide**: `df.pivot_table(index="ts", columns="external_id", values="value")` so each point is a column (one column per `external_id`).
4. **Rename columns** using a **column map** derived from Brick TTL (`resolve_from_ttl`): external refs or labels become the names rules expect (Brick-class-driven mapping).
5. **Add** `timestamp = pd.to_datetime(df["ts"])` for time-based checks.

Under the hood, pandas is doing **grouped aggregation in the pivot** (duplicate `(ts, external_id)` would aggregate), then **aligning** all series to a common `DatetimeIndex` (implicit via the pivot index).

18.3 3. What `RuleRunner.run` does to that DataFrame

`RuleRunner.run` (`open_fdd.engine.runner`):

1. **`result = df.copy()`** — rules never mutate the caller's frame in place (callers can still hold a reference to the original).
2. For **each** rule dict:
   - Derives **`flag_name`** from `flag` or `{name}_flag`.
   - Calls **`_evaluate_rule`** → returns a **boolean `Series`** (fault mask) aligned to `result.index`.
   - Optionally applies a **rolling window** on the mask: `mask.astype(int).rolling(window=rw).sum() >= rw` so a fault must persist for N consecutive samples.
   - Assigns **`result[flag_name] = ...`** as integer 0/1.

So each rule adds **one column** of flags; the frame grows **width-wise**, not row-wise.

18.4 4. How a YAML rule dict becomes pandas expressions

Inside `_evaluate_rule`, Open-FDD branches on `rule["type"]` (e.g. `bounds`, `flatline`, `expression`, …):

- **`column_map` resolution**: For each logical input key, the runner picks a **DataFrame column name**. If the YAML input has a **`brick`** class, the global `column_map` from TTL is consulted first (`brick` → column label), so the **same YAML** can run against different exports as long as TTL maps Brick classes to the right columns.
- **Bounds / thresholds**: Typical pattern is `(series < low) | (series > high)` using **vectorized** comparisons — these are **numpy ufuncs** under pandas, no Python `for` over rows.
- **Expressions**: String expressions may be evaluated in a **restricted eval** context (`open_fdd.engine.checks.check_expression`) with named series bound to `result[col]` — still vectorized.

If a required column is missing, the runner either **raises** or **skips** the rule (`skip_missing_columns=True`), depending on the call site.

18.5 5. Hot reload: YAML vs DataFrame lifetime

`run_fdd_loop` calls **`load_rules_from_dir(rules_path)` every run**, then filters by equipment types from TTL, then **`RuleRunner(rules=rules)`**. There is **no** long-lived compiled rule object on disk—editing YAML affects the **next** scheduled run (or the next manual `POST /run-fdd`). The **DataFrame** exists only for the duration of that run's Python call stack (load → run → persist results).

18.6 6. Mental model checklist

Artifact	In-memory shape	pandas role
Rule YAML files	N/A (on disk)	None until loaded
Loaded rules	`list[dict]`	None
Telemetry window	`DataFrame` (time × points)	pivot, datetime index, column rename
Rule output	Same index, +flag columns	vectorized masks, optional `rolling`
`fault_results` / DB	Written from row-wise `FDDResult`	After flags are computed

For a **minimal** library-side example, use **`pd.read_csv`** + **`RuleRunner`** as in Getting started; sample CSVs and notebooks live under **`examples/AHU/`**. Unit tests: `open_fdd/tests/engine/test_runner.py`.

19 Fault rules for HVAC

20 Fault rules for HVAC

FDD fault rule types, YAML format, and expression cookbook for HVAC and building systems. Rules are **100% Brick-model driven**: inputs reference Brick classes only; column mapping comes from the Brick TTL and Brick external timeseries references (timeseries storage), which Open-FDD embraces at the heart of FDD.

Page	Description
Overview	Rule types, `stack/rules`, hot reload, Brick inputs
Test bench rule catalog	Links to reference YAML under `openclaw/bench/rules_reference/` (AHU FC, chillers, weather, etc.)

21 Contributing

22 Contributing to Open-FDD

First off, thanks for taking the time to contribute!

> **Documentation split:** This site documents the `open-fdd` PyPI engine. Pull requests that touch **Docker, API, React, BACnet, or the data model** should update **open-fdd-afdd-stack docs** as well (or instead), and cross-link from here when the engine API changes.

Open-FDD is in **Alpha**. The most valuable contributions right now are **bug reports** and **FDD rules**—especially from mechanical engineers and building professionals who can add and refine fault-detection rules using the Expression Rule Cookbook. All types of contributions are encouraged.

Phase focus: Alpha emphasizes platform stability, driver implementation beyond BACnet, and API changes for specific integration needs. Beta (planned) will focus more on Brick data modeling (e.g. ASHRAE 223P), mechanical engineering and consulting input into the expression rule cookbook, and better default Grafana dashboards for HVAC analytics. See the Table of Contents for different ways to help and how this project handles them. Please read the relevant section before contributing; it helps maintainers and keeps things smooth for everyone.

> If you like the project but don't have time to contribute, that's fine. Other ways to support it: - Star the repository - Share it with colleagues or at meetups - Refer to Open-FDD in your project's readme or documentation

---

22.1 Table of Contents

- Code of Conduct
- I Have a Question
- I Want To Contribute
- Reporting Bugs
- Suggesting Enhancements
- Contributing Rules (Expression Rule Cookbook)
- Your First Code Contribution
- Improving the Documentation
- Styleguides
- Git workflow (branch → PR → sync)
- Commit Messages
- Join the Project Team

22.2 Code of Conduct

This project expects everyone to be respectful and constructive. By participating, you agree to uphold a positive environment. Please report unacceptable behavior by opening an issue or contacting the repository maintainers.

22.3 I Have a Question

Before asking, please check the documentation and search existing issues to see if your question is already answered.

If you still need help:

- Open an issue.
- Provide as much context as you can (what you're trying to do, what you ran, what happened).
- Include relevant versions: Python, Docker, OS (e.g. Ubuntu, Linux Mint), and Open-FDD commit or release if known.

We'll respond as soon as we can.

22.4 I Want To Contribute

22.4.1 Legal notice

When contributing, you agree that you have authored 100% of the content, have the necessary rights to it, and that your contribution may be provided under the project license (MIT).

22.5 Reporting Bugs

Bug testing is a top priority in Alpha. Good bug reports help us fix issues quickly.

22.5.1 Before submitting a bug report

- Use the latest version (main branch or latest release).
- Confirm the bug is in Open-FDD and not in your environment (e.g. wrong Python version, missing config). Check the documentation and I Have a Question first.
- Search issues to see if the bug is already reported.
- Collect:
  - **Stack trace** (Traceback) if applicable
  - **OS and platform** (e.g. Linux, macOS, Windows; x86, ARM)
  - **Versions**: Python, Docker, Docker Compose, and how you ran Open-FDD (pip, Docker Compose, etc.)

    - **Steps to reproduce** and, if possible, sample input/config
    - Whether you can reproduce it every time and on an older version

22.5.2 How to submit a good bug report

- **Security issues:** Do **not** report security vulnerabilities in public issues. Email the repository owner or open a private security advisory on GitHub.
- Open a new issue. Don't assume it's a bug yet—avoid using the word "bug" in the title until it's confirmed.
- Describe **expected behavior** vs **actual behavior**.
- Provide **reproduction steps** so someone else can recreate the issue. Isolate the problem when possible (e.g. minimal rule YAML, minimal config).
- Paste the information you collected above.

After you file:

- Maintainers will label the issue. If we can't reproduce it, we may ask for more steps and mark it `needs-repro`. Reproducible issues may be marked `needs-fix` and opened for implementation.

22.6 Suggesting Enhancements

Enhancements are tracked as GitHub issues.

22.6.1 Before submitting

- Use the latest version and read the documentation to see if the behavior already exists or can be configured.
- Search issues to see if the enhancement was already suggested; if so, add to that discussion.
- Consider whether the idea fits Open-FDD's scope (edge AFDD, rules, API, BACnet, Grafana). Make a clear case for why it would help most users.

22.6.2 How to submit a good enhancement suggestion

- Use a **clear, descriptive title**.
- Describe the **current behavior** and what you **want instead**, and why.

- Provide **step-by-step detail** where useful; screenshots or example YAML/config can help.
- Explain why this would be useful to the community.

22.7 Contributing Rules (Expression Rule Cookbook)

We especially welcome contributions from mechanical engineers and building professionals who can add or improve FDD rules.

- **Where rules live:** Fault rules overview — put project rules in `stack/rules/` (YAML). The FDD loop reloads them every run; no restart needed.
- **How to write rules:** Expression Rule Cookbook — expression-type rules use YAML with BRICK-style inputs, params, and pandas/NumPy expressions. The cookbook includes AHU-style rules (e.g. GL36-inspired) and patterns you can adapt.
- **What to contribute:**
  - **New rule YAMLs** for common faults (AHU, VAV, plant, sensors) that others can reuse.
  - **Improvements to existing cookbook rules** (thresholds, logic, descriptions).
  - **New cookbook sections or examples** (e.g. for a specific equipment type or standard).

To contribute a rule or cookbook change:

1. Add or edit files under `stack/rules/` for new/updated rules, or under `docs/expression_rule_cookbook.md` (and related docs) for cookbook content.
2. Follow the existing YAML style and BRICK naming used in the cookbook.
3. In a PR, describe the fault the rule targets and any assumptions (e.g. Brick types, units). If it's from a standard (e.g. ASHRAE), note the reference.

22.8 Your First Code Contribution

- Look for issues labeled `good first issue` or `help wanted` (if we've added them).

- The codebase is Python 3.11+, with FastAPI (API), pandas/NumPy (rules), and Docker for the platform. See Getting Started and the README for setup.
- For rule changes, run the test suite and, if possible, the FDD loop locally with sample data before submitting a PR.

22.9 Improving the Documentation

Documentation lives in the `docs/` directory and is published at bbartling.github.io/open-fdd (Just the Docs theme). Improvements are welcome:

- Fix typos, clarify wording, or update steps.
- Add examples or how-tos (e.g. for the Expression Rule Cookbook, Engine-only / IoT, or the AFDD stack how-to index).
- Keep code blocks and links in sync with the codebase.

Building a PDF: To generate a single PDF of the docs (e.g. for offline use or to push to the repo), run `python3 scripts/build_docs_pdf.py`. Requires pandoc and either **weasyprint** (`pip install weasyprint`) or LaTeX. Output is `pdf/open-fdd-docs.pdf` by default; use `--output path` to change it. You can commit and push the PDF if you want it in the repo.

Open a PR with your changes; for large edits, an issue first can help align with maintainers.

After changing README or high-traffic doc links, spot-check the live site (GitHub Pages builds extensionless URLs like `/modeling/llm_workflow`; the same path with a **trailing slash** often 404s because Jekyll emits `page.html`, not `page/index.html`):

```
        BASE=https://bbartling.github.io/open-fdd
        curl -sS -o /dev/null -w "%{http_code}" "$BASE/modeling/llm_workflow"    # expect 200
        curl -sS -o /dev/null -w "%{http_code}" "$BASE/modeling/llm_workflow/"   # often 404 — avoid trailing slash in external links
        curl -sS -o /dev/null -w "%{http_code}" "https://github.com/bbartling/open-fdd/blob/master/pdf/open-fdd-docs.pdf"  # expect 302 or 200
```

22.10 Styleguides

- **Python:** We use Black for formatting. Run `black` on changed files before committing.
- **YAML:** Match the style in `stack/rules/` and the expression rule cookbook (indentation, key order).
- **Markdown:** Use clear headings and lists; link to existing docs where relevant.

22.11 Git workflow (branch → PR → sync)

Create the branch **before** committing so that after the PR is merged, `git pull` on `main`/`master` fast-forwards and you avoid divergent-branch errors (protected `main`/`master`).

1. **Create a branch**

```
git checkout -b feature/short-name
```

2. **Stage, commit, and push**

```
git add .
git commit -m 'short note describing the change'
git push -u origin feature/short-name
```

3. **Open a PR manually on GitHub** from `feature/short-name` into `main` (or `master`). Create the PR in the GitHub UI; we do not use a CLI for this step.

4. **After the PR is merged** — fetch prune, switch to default branch, and pull to sync:

```
git fetch --prune
git checkout main
git pull
```

   Use `master` instead of `main` if that is your default branch. If you ever committed on `main`/`master` before creating the branch, local and remote can diverge after the merge; run `git reset --hard origin/main` (or `origin/master`) to match the remote.

22.12 Commit Messages

- Use a clear, short summary line. Optionally add a body with details.
- Reference issues/PRs when relevant (e.g. `Fix bounds rule when input is all NaN (#123)`).

22.13 Join the Project Team

If you're interested in ongoing maintenance or a larger role, say so in an issue or reach out to the maintainers. We're especially interested in folks who can triage bugs, review rule contributions, or improve docs and the cookbook.

22.14 Attribution

This contributing guide was inspired by the contributing.md project. You don't need to pay to create or use a CONTRIBUTING file—it's just a markdown file in your repo that helps contributors and maintainers.

23 Developer guide

24 Developer guide

The **`open-fdd`** package is the **rules engine**: YAML rule configs, **`RuleRunner`** on **pandas** `DataFrame`s, and helpers to build **`column_map`** from Brick TTL or manifests. Everything on this page is **library scope** only.

24.1 Exported symbols

Typical imports

```
from open_fdd import RuleRunner, resolve_from_ttl
```

Full engine surface (`open_fdd.engine` — same package, explicit submodule):

Symbol	Role
`RuleRunner`	Load rules from a directory or list of dicts; `run(df, ...)` adds fault flag columns
`load_rule`, `bounds_map_from_rule`	Load one YAML file; extract bounds map for analytics
`resolve_from_ttl`	Brick TTL → column mapping (requires **`open-fdd[brick]`**)
`BrickTtlColumnMapResolver`, `ManifestColumnMapResolver`, `FirstWinsCompositeResolver`, `load_column_map_manifest`	Build `column_map` for `RuleRunner.run`

Source: `open_fdd/engine/` on GitHub. Behavior and parameters are documented in **docstrings** on `RuleRunner.run` and resolver classes.

24.2 Repository layout

Path	Role
`open_fdd/engine/`	Runner, checks, expression evaluation, column-map resolvers, Brick helpers
`open_fdd/schema/`, `open_fdd/reports/`	Shared models and reporting helpers
`open_fdd/tests/`	Pytest suites
`examples/`	Notebooks and resolver workshops

24.3 Setup and tests

```
pip install -e ".[dev]"
pytest open_fdd/tests/ -v --tb=short
```

TESTING.md

24.4 Extending the engine

- New rule **types** or runner behavior → `open_fdd/engine/`; keep on-disk YAML shape stable when possible.
- New **public** symbols → export from `open_fdd/engine/__init__.py` (and top-level `open_fdd/__init__.py` when appropriate), then document on this site.

24.5 Other repository

The **open-fdd-afdd-stack** repo documents a separate on-prem **product** that **consumes** this engine (installed from PyPI). For that product, see its **documentation site** — it is not covered here.

25 Appendix

26 Appendix

Engine repo reference for contributors.

Page	Description
Developer guide	Layout of `open_fdd/`, tests, how to extend the engine

Platform REST, TTL import semantics, operations, and security are documented on **open-fdd-afdd-stack docs** (API reference, technical reference).